Reporte de práctica del driver de relevador

Introducción

Cuando controlamos un dispositivo desde Arduino, escribir HIGH o LOW parece suficiente. El problema aparece cuando cada componente responde de manera distinta o cuando debemos limitar qué tan rápido cambia. Si esas condiciones quedan repartidas por el programa, es más fácil cometer un error al agregar nuevas funciones.

En esta práctica se plantea una clase Relevador que concentra esas decisiones. El motor, conectado a un L298N, representa una carga activa en alto. El LED representa una carga activa en bajo. La idea es darles las mismas órdenes desde el programa y dejar que cada objeto aplique el nivel que le corresponde.

Objetivo

Desarrollar una clase en C++ que controle el encendido y apagado de dos actuadores, encapsule su estado y aplique un intervalo mínimo de 1000 ms entre cambios, sin utilizar delay().

Fundamento

El driver de software traduce órdenes como encender() y apagar() a una salida eléctrica. El L298N es la etapa de potencia: recibe una señal del Arduino y conmuta la energía que proviene del portapilas. Aunque en esta práctica se use como un interruptor electrónico, no es un relevador mecánico ni proporciona aislamiento galvánico.

El encapsulamiento permite que el estado y la restricción de tiempo se modifiquen únicamente dentro de la clase. millis() permite medir el tiempo transcurrido sin detener el programa, de modo que siguen disponibles la recepción de comandos y la consulta de estados.

Materiales

Cantidad	Material
1	Arduino UNO R4 WiFi y cable USB-C.
1	Módulo L298N y motorreductor TT de 3 a 6V.
2	Pilas 18650 con portapilas en serie.
1	LED y resistencia en serie; guía: 220 Ω.
1	Protoboard y cables jumper necesarios.

Placa y resistencia utilizadas realmente: [completar]. Revisar la corriente del LED antes de energizar; el análisis del valor de 220 Ω se encuentra en el documento de diagramas.

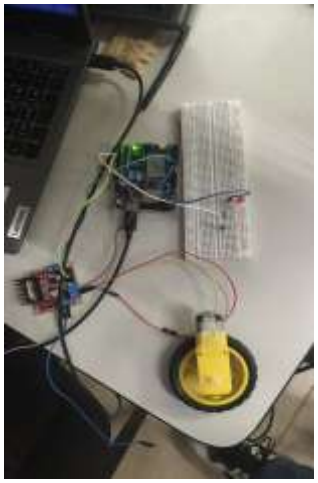
Conexiones del sistema

D7 controla IN1 del L298N e IN2 se conecta a GND. El motor se conecta entre OUT1 y OUT2. El portapilas alimenta la entrada de potencia del módulo y se comparte tierra con el Arduino. La terminal +5V del L298N no se conecta al Arduino. Los jumpers ENA y 5V-EN se revisan según la guía y el módulo disponible.

El ánodo del LED se conecta a 5V y su cátodo llega a D6 mediante una resistencia. Cuando D6 está en LOW, la corriente puede circular desde 5V hacia el pin y el LED enciende. Con D6 en HIGH, la diferencia de potencial deja de ser suficiente para encenderlo.

Preparación de la práctica

El montaje debe hacerse con el USB y el portapilas desconectados. Antes de energizar se revisan la polaridad, la tierra común, la corriente del LED y la tensión aplicada al motor. La guía solicita que el docente revise el circuito. Evidencia del montaje y revisión:



Simulación

La guía contempla hardware físico y Tinkercad. Si se emplea una placa equivalente en simulación, registrar la diferencia respecto del UNO R4 WiFi. Una simulación equivalente puede servir para comprobar la lógica, pero no verifica la corriente real ni el comportamiento del montaje físico. Enlace y configuración utilizados:

Diseño de la solución

La clase contiene el pin, el tipo de activación, el estado lógico y el instante del último cambio. El motor se declara como Relevador motor(7, false) y el LED como Relevador led(6, true). La configuración se concentra en esos objetos para que el resto del programa utilice las mismas operaciones.

iniciar() deja cada salida apagada. encender() y apagar() comprueban si la orden implica una transición. Si ya transcurrió un segundo, actualizan el estado y la salida; en caso contrario devuelven false. estaEncendido() permite consultar el estado lógico almacenado.

Manejo del tiempo

El tiempo se calcula mediante la resta del valor actual de millis() y el instante registrado. No se realiza una espera activa ni se usa delay(). Cada objeto conserva su contador, por lo que una conmutación del motor no impide controlar el LED cuando este ya cumple su propio intervalo.

Se adopta un criterio explícito: la primera conmutación espera un segundo desde iniciar(). Repetir una orden que pide el estado actual no cambia el pin ni reinicia el tiempo. Este comportamiento evita que una orden repetida retrase una transición válida.

Interacción por Monitor Serie

La comunicación se configura a 9600 baudios. M1 y M0 controlan el motor; L1 y L0 controlan el LED; E consulta ambos estados. Se envía un comando por línea, en mayúsculas. El sketch recibe los caracteres disponibles y procesa la orden al recibir el salto de línea.

Estado seguro

Al ejecutarse setup(), los dos objetos se inicializan apagados. Esto reduce el riesgo de que una carga se active al comenzar el programa. El software no garantiza por sí solo lo que sucede antes de setup(), cuando los pines pueden estar en alta impedancia. Para asegurar todo el periodo de reinicio se necesita una polarización eléctrica adecuada.

Los diagramas de estados y conexiones acompañan el proyecto en la carpeta diagramas/. El código completo está en codigo/DriverRelevador/.

Pruebas y registro de resultados

Completar la columna de resultados después de ejecutar las pruebas. El comportamiento esperado funciona como criterio de comparación; no representa una medición realizada.

Prueba	Resultado esperado	Resultado observado
Inicio y consulta E	Motor y LED apagados tras iniciar().	[completar]
M1 y M0 con espera	Motor cambia y responde al comando.	[completar]
L1 y L0 con espera	LED cambia con lógica invertida.	[completar]
M1 y luego M0 antes de 1 s	M0 rechazado; el motor sigue encendido.	[completar]
M0 al cumplir 1 s desde M1	Apagado aceptado.	[completar]
Repetir M1 estando encendido	Sin conmutación ni reinicio del contador.	[completar]
RESET con motor encendido	Ambos apagados al ejecutarse iniciar().	[completar]
LED configurado activo en alto	Comportamiento físico invertido.	[completar]

Evidencias

Agregar una fotografía del circuito y capturas de las respuestas del Monitor Serie. Para probar el rechazo, el segundo comando debe llegar antes de un segundo desde el último cambio aceptado. Si enviarlo a mano tarda demasiado, pegar dos líneas seguidas, M1 y M0, después de cumplir la espera inicial.

Evidencia del montaje: [archivo o enlace].
Evidencia de órdenes y rechazo: [archivo o enlace].
Observación del reinicio y de la inversión del LED: [completar].

Dificultades y ajustes

Problemas encontrados, causa identificada y cambio aplicado: [completar con lo ocurrido durante la práctica].

Preguntas de reflexión

1 Por qué el LED se enciende con LOW

El ánodo está conectado a 5V. Al colocar D6 en LOW, la corriente circula de 5V al LED, pasa por la resistencia y entra al pin hacia tierra. Con HIGH, ambos extremos quedan cerca de 5V y el LED deja de tener la tensión necesaria para encenderse. Por eso la clase debe distinguir el estado lógico del nivel aplicado.

2 Ventaja de guardar la regla de tiempo en la clase

La regla se aplica desde cualquier parte del programa que use encender() o apagar(). Si dependiera de loop(), otra función podría olvidarse de comprobarla. Además, cada objeto conserva su propio tiempo y no necesitamos repetir la condición para el motor y el LED.

3 Por qué un relevador mecánico no se usa para PWM

Sus contactos necesitan tiempo para moverse. El PWM utilizado para regular un motor exige cambios mucho más rápidos de lo que puede seguir un relevador mecánico y provocaría desgaste, rebotes y arcos en los contactos. Para esa tarea se utilizan dispositivos electrónicos adecuados. El L298N puede trabajar con PWM en una aplicación diseñada para ello, pero aquí solo se usa para encendido y apagado.

4 Por qué se utiliza apagado como estado seguro

En este montaje conviene evitar que el motor comience a girar sin una orden. Sin embargo, el estado seguro depende del proceso. En un sistema de extracción que deba continuar funcionando si falla el control, un contacto NC puede mantener alimentado el ventilador mientras la fuente de potencia siga disponible. NC significa cerrado cuando la bobina está desenergizada; no garantiza funcionamiento si también se pierde la alimentación de la carga.

5 Por qué se necesita tierra común

El L298N interpreta la señal de D7 respecto de su propia tierra. Compartir GND establece una referencia para reconocer HIGH y LOW y permite el retorno de la corriente de señal. Esto no implica alimentar el motor desde el Arduino. En un diseño con aislamiento galvánico real, la conexión entre tierras depende de la arquitectura.

6 Qué cambia al usar un módulo de relevador real

Si el módulo acepta una entrada digital compatible, se ajustan el pin y el argumento activoEnBajo según su funcionamiento. La interfaz de la clase y los comandos pueden conservarse. En el hardware cambian la alimentación, el cableado de la carga y la elección entre COM, NO y NC. La bobina de un relevador suelto nunca se conecta directamente a un GPIO.

Prueba de inversión del LED

Si se construye por error como Relevador led(6, false), encender() aplica HIGH y el LED físico permanece apagado. apagar() aplica LOW y el LED enciende. Incluso al inicializar, la clase cree que lo dejó apagado, pero el circuito se activa. Esta prueba muestra por qué estaEncendido() informa un estado lógico y no una confirmación física.

Conclusión

El diseño de esta práctica permite relacionar la programación orientada a objetos con el control de un dispositivo. La clase no solo evita repetir instrucciones: también concentra las condiciones que deben cumplirse antes de cambiar una salida. Al separar el estado lógico del nivel eléctrico, las mismas órdenes sirven para dos actuadores con comportamientos distintos.

El uso de millis() permite comprobar el intervalo sin detener la lectura de comandos. También queda claro que una medida de software tiene límites: iniciar las salidas apagadas no reemplaza una conexión correcta ni garantiza el estado eléctrico durante todo el reinicio. La revisión del montaje y las pruebas físicas son necesarias para confirmar el funcionamiento.